

**NANYANG
TECHNOLOGICAL
UNIVERSITY**

SINGAPORE

SC4003/CE/CZ4046: Intelligent Agents Assignment 1 Report

Name: Ivan Soji Varghese

Matriculation ID: U2220650L

Date: March 22, 2026

Introduction

The objective here is to use Markov Decision Process (MDP) to create setup with known states, actions, transitions and rewards. Suppose in this maze, where actions are stochastic, and each state has a reward, the goal of the agent shall be to find the best long-term strategy (policy). Therefore, it must act optimally in a noisy environment.

Some ground rules:

- The actions are stochastic, but 80% of the time it shall follow its intended direction, and 10% of the time it shall either move left or right of its intended direction. If there's a wall, it shall not move. These actions are not deterministic.
- Rewards:
 - White squares: -0.05 (a minor penalty for moving)
 - Green squares: +1 (towards optimum)
 - Red squares: -1 (to avoid)
 - No terminal states. Shall run/move forever.
- Discount factor of 0.99. Future rewards matter quite much.

Part 1

Find optimal policy and utilities of all non-wall states using value and policy iterations.

Value Iteration

To iteratively update:

$$U(s) = R(s) + \gamma * \max_a \sum P(s'|s,a) * U(s')$$

Where:

- $U(s)$: Utility value of a state. Includes both immediate and future rewards
- $R(s)$: Immediate reward for being in a current state.
- γ : Discount factor
- $\max_a \sum P(s'|s,a) * U(s')$: Choose the best action (optimal) - picking the action which gives the highest value. It shall do this by calculating the expected future value of all next states, their probabilities, multiplication of those state values + summation.

For each state:

- Try all actions
- Choose the best expected outcome

To output:

- Optimal policy
- Utilities of all states
- Plot of utility and iterations

Development

Programming language chosen for development shall be Python.

First, we shall start with the creation of a maze, using numbers for rewards and 'None' for walls; to use a 2D grid - a simple 2D array.

Take note the code blocks below are mere pseudocodes that were used to plan structuring and architecture of the program. Final code differs.

```
grid = [  
    [-0.05, -0.05, -0.05, 1.0],  
    [-0.05, None, -0.05, -1.0],  
    [-0.05, -0.05, -0.05, -0.05]  
]
```

Every non-wall cell is a valid state, and every state already stores its reward.

At this stage, a small helper that lists all valid states is created:

```
states = []  
for r in range(len(grid)):  
    for c in range(len(grid[0])):  
        if grid[r][c] is not None:  
            states.append((r, c))  
  
print(states)
```

Here, we need to know which are the valid states, and what rewards each state has.

Define Actions and Movement

Now the four actions:

```
ACTIONS = ["U", "D", "L", "R"]
```

```
ACTION_DELTAS = {  
    "U": (-1, 0),  
    "D": (1, 0),  
    "L": (0, -1),  
    "R": (0, 1),  
}
```

Now a function that attempts a move:

```
def is_valid(grid, r, c):  
    return (  
        0 <= r < len(grid) and  
        0 <= c < len(grid[0]) and  
        grid[r][c] is not None  
    )
```

```
def move(grid, state, action):  
    r, c = state  
    dr, dc = ACTION_DELTAS[action]  
    nr, nc = r + dr, c + dc  
    if is_valid(grid, nr, nc):  
        return (nr, nc)
```

```
return state
```

If agent would hit a wall, it stays in the same place – so move() must enforce that.

```
print(move(grid, (0, 0), "R")) # should move right
print(move(grid, (1, 0), "R")) # if wall, should stay
```

Define the Stochastic Transition Model

If the agent chooses "U":

- 0.8 goes up
- 0.1 slips left
- 0.1 slips right

So the right-angle mapping:

```
RIGHT_ANGLES = {
    "U": ["L", "R"],
    "D": ["L", "R"],
    "L": ["U", "D"],
    "R": ["U", "D"],
}
```

Now the transition function:

```
def transitions(grid, state, action):
    outcomes = {}
    possibilities = [
        (0.8, action),
        (0.1, RIGHT_ANGLES[action][0]),
        (0.1, RIGHT_ANGLES[action][1]),
    ]
    for prob, actual_action in possibilities:
        next_state = move(grid, state, actual_action)
        outcomes[next_state] = outcomes.get(next_state, 0) + prob
    return list(outcomes.items())
```

2 different failed moves may both leave the agent in the same state. Probabilities must be merged.

```
print(transitions(grid, (0, 0), "U"))
print(transitions(grid, (1, 0), "R"))
```

These shall add to 1.

Utilities

Each state has a utility value, usually initialized to 0:

```
utilities = {}
for s in states:
    utilities[s] = 0.0
```

Utility represents how good it is to be in that state, considering long-term future rewards – not just immediate reward.

Compute Expected Utility of One Action

Before value iteration, the helper that scores one action at one state:

```
def expected_utility(grid, state, action, U):
    total = 0.0
    for next_state, prob in transitions(grid, state, action):
```

```

    total += prob * U[next_state]
return total

```

This computes: $\sum P(s'|s,a) \cdot U(s')$ – the average future utility expected from choosing this action.

Then a optimal action helper:

```

def best_action_and_value(grid, state, U):
    best_action = None
    best_value = float("-inf")
    for action in ACTIONS:
        val = expected_utility(grid, state, action, U)
        if val > best_value:
            best_value = val
            best_action = action
    return best_action, best_value

```

At the start, all utilities are 0, so all actions will likely tie.

Implement Value Iteration

The Bellman optimality update: utility of state = immediate reward + γ × best expected future utility.

```

def value_iteration(grid, states, gamma=0.99, epsilon=1e-6):
    U = {s: 0.0 for s in states}
    history = [U.copy()]
    while True:
        U_new = {}
        delta = 0.0
        for s in states:
            reward = grid[s[0]][s[1]]
            _, best_future = best_action_and_value(grid, s, U)
            U_new[s] = reward + gamma * best_future
            delta = max(delta, abs(U_new[s] - U[s]))
        U = U_new
        history.append(U.copy())
        if delta < epsilon * (1 - gamma) / gamma:
            break
    return U, history

```

U_new instead of updating in place as value iteration should use the previous iteration's utilities consistently across all states. Updating in-place changes the algorithm.

Extract the Policy from Utilities

Once value iteration converges, compute the best action at each state:

```

def extract_policy(grid, states, U):
    policy = {}
    for s in states:
        best_action, _ = best_action_and_value(grid, s, U)
        policy[s] = best_action
    return policy

```

```

U_vi, history_vi = value_iteration(grid, states)
policy_vi = extract_policy(grid, states, U_vi)

```

At this point you already have half the assignment done – utilities, optimal policy, and utility-

vs-iteration plot. That is the full Part 1 value iteration component.

Display Utilities and Policy

Utility Printer

```
def print_utilities(grid, U):
    for r in range(len(grid)):
        row = []
        for c in range(len(grid[0])):
            if grid[r][c] is None:
                row.append("#####")
            else:
                row.append(f'U[(r,c)]:{6.3f}')
        print(" ".join(row))
```

Policy Printer

```
def print_policy(grid, policy):
    for r in range(len(grid)):
        row = []
        for c in range(len(grid[0])):
            if grid[r][c] is None:
                row.append(" # ")
            else:
                row.append(f' {policy[(r,c)]} ')
        print(" ".join(row))
```

Plot Utility Estimates Over Iterations

We need utility estimates as a function of number of iterations. Store history in value iteration and plot selected states:

import matplotlib.pyplot as plt

```
def plot_history(history, tracked_states):
    x = range(len(history))
    for s in tracked_states:
        y = [U[s] for U in history]
        plt.plot(x, y, label=str(s))
    plt.xlabel("Iteration")
    plt.ylabel("Utility")
    plt.title("Utility estimates over iterations")
    plt.legend()
    plt.grid(True)
    plt.show()
```

Implement Policy Iteration

Policy iteration has 2 alternating parts: evaluate current policy, then improve policy.

Start with Arbitrary Policy

policy = {s: "U" for s in states}

Policy Evaluation

Computes utilities assuming the agent follows the current fixed policy:

```
def policy_evaluation(grid, states, policy, gamma=0.99, epsilon=1e-6):
    U = {s: 0.0 for s in states}
    while True:
        U_new = {}
        delta = 0.0
        for s in states:
            reward = grid[s[0]][s[1]]
```

```

        action = policy[s]
        future = 0.0
        for next_state, prob in transitions(grid, s, action):
            future += prob * U[next_state]
        U_new[s] = reward + gamma * future
        delta = max(delta, abs(U_new[s] - U[s]))
    U = U_new
    if delta < epsilon * (1 - gamma) / gamma:
        break
    return U

```

Evaluating the utility of the currently chosen action only, no maximum choosing

Policy Improvement

```

def policy_iteration(grid, states, gamma=0.99, epsilon=1e-6):
    policy = {s: "U" for s in states}
    history = []
    while True:
        U = policy_evaluation(grid, states, policy, gamma, epsilon)
        history.append(U.copy())
        stable = True
        new_policy = {}
        for s in states:
            old_action = policy[s]
            best_action, _ = best_action_and_value(grid, s, U)
            new_policy[s] = best_action
            if best_action != old_action:
                stable = False
        policy = new_policy
        if stable:
            break
    return U, policy, history

```

Stop condition: stop when policy no longer changes. That means the policy is stable and optimal.

Compare the Two Algorithms (run on maze)

```

U_vi, history_vi = value_iteration(grid, states)
policy_vi = extract_policy(grid, states, U_vi)

```

```

U_pi, policy_pi, history_pi = policy_iteration(grid, states)

```

From the assignment, we can make the map:

```

grid = [
    [ 1.0, None, 1.0, -0.05, -0.05, 1.0],
    [-0.05, -1.0, -0.05, 1.0, None, -1.0],
    [-0.05, -0.05, -1.0, -0.05, 1.0, -0.05],
    [-0.05, -0.05, -0.05, -1.0, -0.05, 1.0],
    [-0.05, None, None, None, -1.0, -0.05],
    [-0.05, -0.05, -0.05, -0.05, -0.05, -0.05],
]

```

```

start_state = (3, 2)

```

Now moving on to check:

- the final policies
- the utilities
- what is converging

Results:

Utilities:

100.000	#####	95.020	93.838	92.605	93.284
98.381	95.868	94.516	94.367	#####	90.874
96.922	95.549	93.256	93.134	93.059	91.741
95.514	94.402	93.173	91.071	91.761	91.835
94.261	#####	#####	#####	89.494	90.503
92.872	91.652	90.447	89.257	88.499	89.223

Policy:

↑	#	←	←	←	↑
↑	←	←	←	#	↑
↑	←	←	↑	←	←
↑	←	←	↑	↑	↑
↑	#	#	#	↑	↑
↑	←	←	←	↑	↑

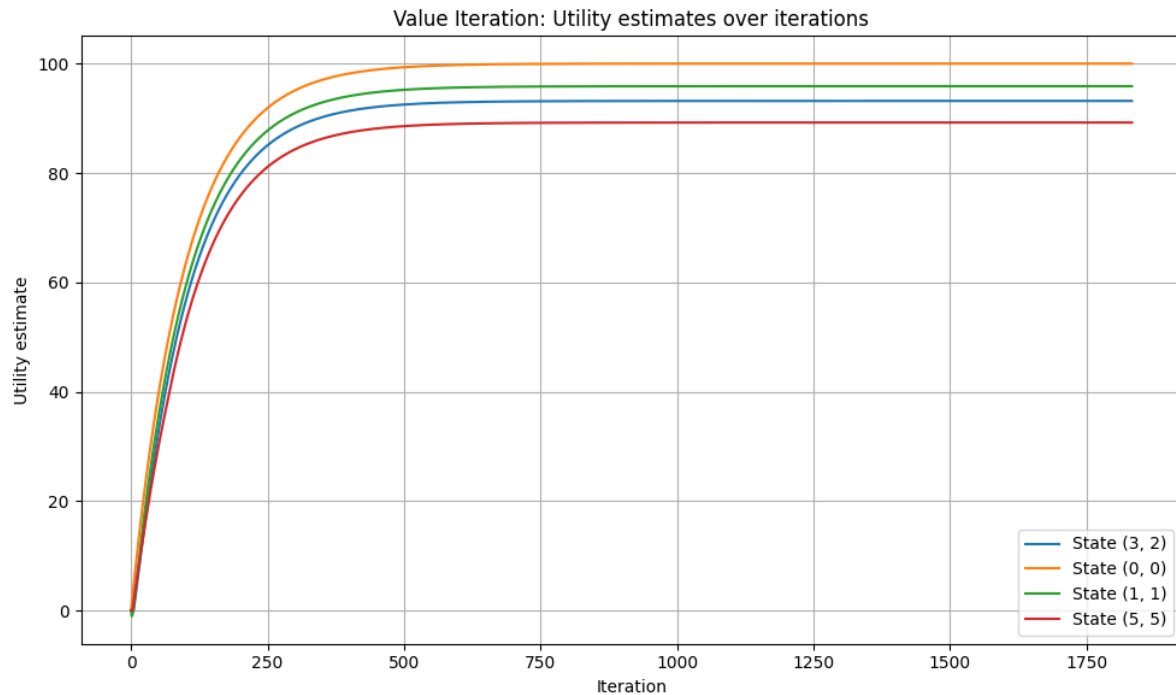
Start state: (3, 2)

Start utility (VI): 93.173047

Best action from start (VI): L

Result

Running value iteration on the 6×6 maze produces the utility values and policy shown above. As expected, utilities are highest in cells close to the +1 rewards, peaking at 100.000 for the green squares themselves. The resulting policy mostly steers the agent upward and to the left, which makes intuitive sense – those directions lead toward the rewarding regions while keeping the agent away from walls and the -1 penalty squares. Starting from state (3,2), the computed utility comes out to 93.173047 and the recommended action is to move left. Given the stochastic nature of the transitions, this suggests that heading left gives the agent the best shot at accumulating long-term reward from that position.



The utility curves from value iteration are pretty well-behaved – all the tracked states rise smoothly without any erratic jumps. In the early iterations, utilities climb quickly as the rewards start spreading outward through the grid. Things slow down as the algorithm progresses, with each update changing the values less and less until the curves essentially flatten out. States that sit close to the +1 cells naturally end up with higher final utilities, while states further away settle at lower values. The convergence takes a fair number of iterations overall, which is largely down to the high discount factor of $\gamma = 0.99$ – since future rewards are barely discounted, the agent has to look a long way ahead, and that takes more iterations to fully propagate through.

Function of the number of iterations:

This is tracking how utilities change over iterations.

For a fixed state $U(s) = f(\text{iteration})$

As per the graph, each line represents a function:

- Blue line $\rightarrow f(k) = U_k(3,2)$
- Orange line $\rightarrow f(k) = U_k(0,0)$
- Green line $\rightarrow f(k) = U_k(1,1)$
- Red line $\rightarrow f(k) = U_k(5,5)$

So plotting becomes:

Iteration (k) $\rightarrow$ utility estimate $U_k(s)$

Each iteration check how many Bellman updates have been done, and how the utility improves over the updates.

The graph plots how the utility estimate of each tracked state changes from one iteration to the next. For any given state, this is just the value the algorithm has assigned to it after k Bellman updates – so each curve traces that value from the initial estimate of 0 all the way through to convergence. The four curves shown correspond to states (3,2), (0,0), (1,1), and

(5,5), and together they give a good picture of how different parts of the grid get "figured out" at different rates as the algorithm runs.

Policy Iteration

Repeats on policy evaluations (of current) and updating (improving) policies using those utilities. To be done until stabilization.

To output:

- Optimal policy
- Utilities of all states
- Plot of utility and iterations

Results:

Utilities:

100.000	#####	95.020	93.838	92.605	93.284
98.381	95.868	94.516	94.367	#####	90.874
96.922	95.549	93.256	93.134	93.059	91.741
95.514	94.402	93.173	91.071	91.761	91.835
94.261	#####	#####	#####	89.494	90.503
92.872	91.652	90.447	89.257	88.499	89.223

Policy:

↑	#	←	←	←	↑
↑	←	←	←	#	↑
↑	←	←	↑	←	←
↑	←	←	↑	↑	↑
↑	#	#	#	↑	↑
↑	←	←	←	↑	↑

Start utility (PI): 93.173047

Best action from start (PI): L

===== COMPARISON: VI vs PI

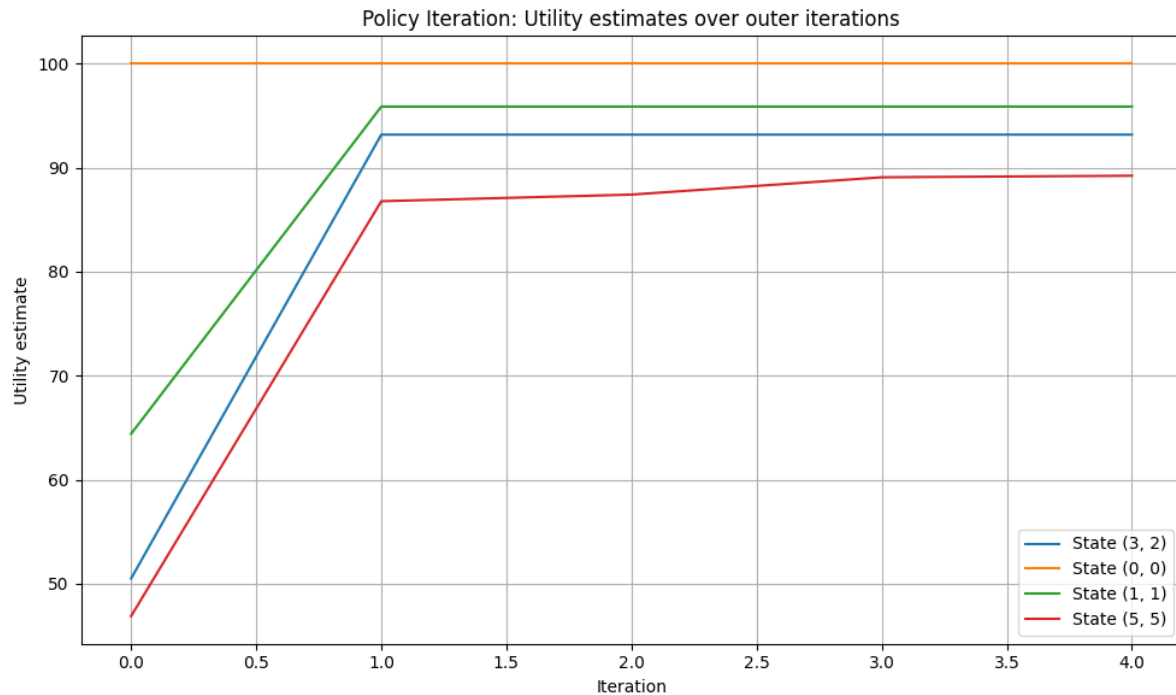
Policies identical? True

Utilities close? True

Maximum utility difference: 1.4210854715202004e-14

Value iteration iterations: 1833

Policy iteration outer iterations: 5



For policy iteration, the horizontal axis represents outer iterations rather than individual Bellman sweeps. Each point corresponds to one full round of policy evaluation followed by policy improvement. This is why the policy iteration graph contains only a few large jumps before stabilizing. The plot here is over outer iterations, and not every internal utility evaluation.

Comparison between VI and PI

Both algorithms end up producing the same optimal policy and very close utility values across the board, which is a good sign that the implementation is correct. The two methods go about things quite differently though – value iteration just keeps applying the Bellman update until things settle, while policy iteration locks in a policy, evaluates it fully, then improves it, and repeats. Policy iteration tends to need far fewer outer loops to stabilise, though each of those loops does more work under the hood. Either way, both converge to the same answer, which is exactly what the theory predicts.

More technically, value iteration and policy iteration produced identical final policies for all non-wall states. Their utility estimates were also effectively identical, with a maximum absolute difference of only 1.42×10^{-14} , which is negligible and attributable to numerical precision. In terms of convergence behavior, value iteration required 1833 utility-update iterations, whereas policy iteration converged in only 5 outer iterations. This confirms the expected theoretical behavior: policy iteration reaches the same optimal solution with fewer outer loops, although each outer loop includes a full policy evaluation step.

Although the immediate rewards are only +1, -1, and -0.05, the resulting utilities are much larger because the maze has no terminal states and the discount factor is high ($\gamma = 0.99$), so the utility reflects accumulated long-run discounted reward over an infinite horizon.

Part 2

Create a maze to run developed algorithm to answer some key questions.

Questions to answer:

- How does number of states affect convergence?
- How does complexity affect convergence?
- What's the largest environment that still works?

Created maze:

To test the algorithm effectiveness: shall make a maze larger than given in Part 1, probably more obstacles (walls) and negative loss areas (-1 squares). To try the below, an 8 by 8 maze:

```
def build_custom_grid():
    grid = [
        [-0.05, -0.05, -0.05, 1.0, None, -0.05, -0.05, 1.0],
        [-0.05, None, -1.0, -0.05, None, -0.05, -1.0, -0.05],
        [-0.05, -0.05, -0.05, -0.05, -0.05, -0.05, -0.05, -0.05],
        [1.0, None, -0.05, None, -1.0, None, -0.05, -0.05],
        [-0.05, -0.05, -0.05, -0.05, -0.05, -0.05, None, 1.0],
        [-1.0, None, None, -0.05, None, -0.05, -0.05, -0.05],
        [-0.05, -0.05, -0.05, -1.0, -0.05, -0.05, None, -0.05],
        [-0.05, 1.0, -0.05, -0.05, -0.05, -1.0, -0.05, -0.05],
    ]
    start_state = (7, 0)
    return grid, start_state
```

Results (following same format as Part 1):

Part 2: VI

Utilities:

82.479	83.720	84.840	86.231	#####	83.556	84.808	86.202
81.389	#####	82.789	84.840	#####	82.482	82.730	84.808
80.403	81.149	82.237	83.405	82.023	81.550	82.364	83.543
81.107	#####	81.149	#####	79.753	#####	81.390	82.324
79.903	78.844	79.809	78.623	78.639	78.191	#####	82.830
77.660	#####	#####	77.581	#####	79.185	80.521	81.601
76.653	76.845	75.939	75.569	76.837	77.993	#####	80.521
76.944	78.016	76.866	75.815	75.861	76.264	78.274	79.325

Policy:

→	→	→	↑	#	→	→	↑
↑	#	↑	↑	#	↑	↑	↑
↑	→	→	↑	←	↑	→	↑
←	#	↑	#	↑	#	↑	↑
↑	←	↑	←	↑	↓	#	←
↑	#	#	↑	#	→	→	↑
↑	↓	←	↑	→	↑	#	↑
→	↓	←	←	↑	→	→	↑

Start state: (7, 0)

Start utility (VI): 76.944481

Best action from start (VI): R

Part 2: PI

Utilities:

82.479	83.720	84.840	86.231	#####	83.556	84.808	86.202
81.389	#####	82.789	84.840	#####	82.482	82.730	84.808
80.403	81.149	82.237	83.405	82.023	81.550	82.364	83.543
81.107	#####	81.149	#####	79.753	#####	81.390	82.324
79.903	78.844	79.809	78.623	78.639	78.191	#####	82.830
77.660	#####	#####	77.581	#####	79.185	80.521	81.601
76.653	76.845	75.939	75.569	76.837	77.993	#####	80.521
76.944	78.016	76.866	75.815	75.861	76.264	78.274	79.325

Policy:

```
→ → → ↑ # → → ↑
↑ # ↑ ↑ # ↑ ↑ ↑
↑ → → ↑ ← ↑ → ↑
← # ↑ # ↑ # ↑ ↑
↑ ← ↑ ← ↑ ↓ # ←
↑ # # ↑ # → → ↑
↑ ↓ ← ↑ → ↑ # ↑
→ ↓ ← ← ↑ → → ↑
```

Start utility (PI): 76.944481

Best action from start (PI): R

Compare PI and VI

Policies identical? True

Utilities close? True

Maximum utility difference: 3.2599700716673397e-11

Value iteration iterations: 1818

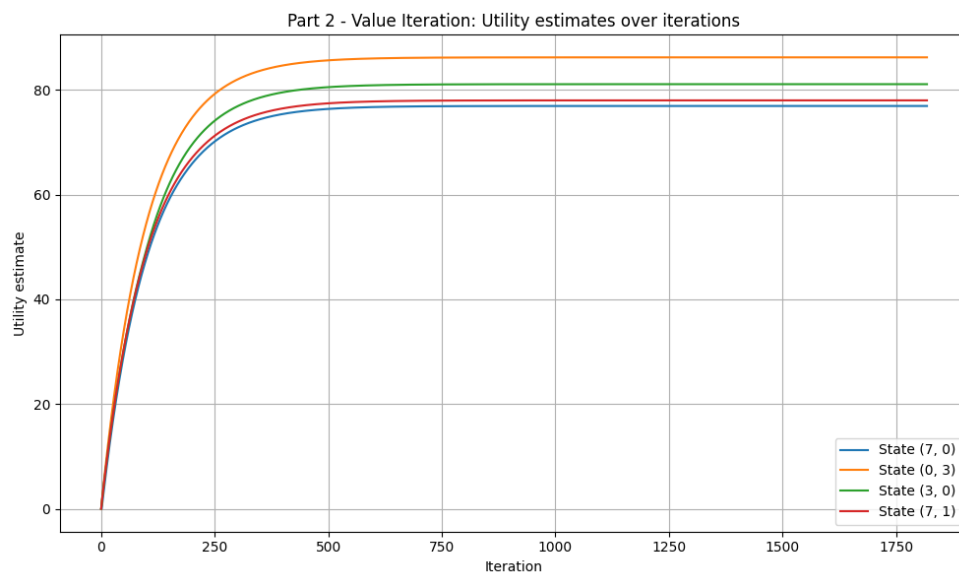
Policy iteration outer iterations: 5

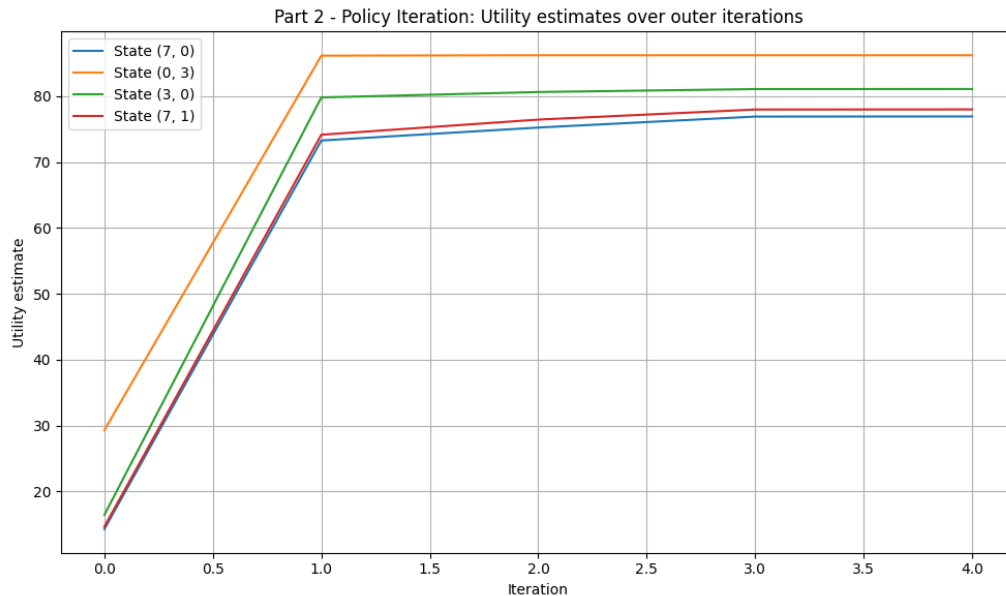
Part 2 summary

Number of non-wall states in custom maze: 53

Value iteration iterations: 1818

Policy iteration outer iterations: 5





To compare between Part 1 and 2:

Utilities and policies are identical. Start state utility at 76.944481. Action is R (or right). Difference in values is very close to 0. Convergence behaviour - VI: 1818 iterations, PI: 5 outer iterations, like Part 1.

For the VI graph: utilities grow gradually and their convergence is slow as $\gamma = 0.99$. For the PI graph: stabilization again by iteration 4-5.

Some observations:

- Part 2 start utility is lower than Part 1, probably as more walls leads to less paths.
- Even with a bigger, more complex maze - Part 2 converges in 5 iterations still. Good efficiency.

To summarize, the utility values are generally lower, reflecting the increased difficulty of navigating the environment. The presence of more walls and negatively rewarded states forces the agent to take longer and safer paths, reducing the overall expected long-term reward.

In terms of convergence behaviour, VI required 1818 iterations to converge, which is similar to the original environment despite the increased number of states. This suggests that convergence is influenced not only by the number of states but also by the structure of the environment. PI again converged in only 5 outer iterations, demonstrating its efficiency even in more complex environments.

The plots show these differences as well. VI shows gradual convergence, while PI exhibits large jumps between iterations due to full policy evaluation steps. Overall, it demonstrates that increasing the size and structural complexity of the environment could lead to more intricate policies and lower utility values, while probably maintaining the same optimal solution across both algorithms.

In terms of scalability, it can be said that the environment shall be a little more complex. Because with these algorithms, what matters is not how complexity increases but rather time computation. As complexity of the maze increases with more walls and larger grids, VI becomes expensive as utility information shall need to propagate across more states over more iterations. PI remains efficient with outer iterations, but its policy evaluation also grows more as the grid expands in size. So, in general, the time required to compute optimal policy becomes the main constraint.